MEDICINE TRACEABILITY SYSTEM USING HYPERLEDGER FABRIC

A PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY IN

COMPUTER ENGINEERING

JMI
Department of Computer Engineering

Under the Supervision of:

Dr. Zeba Anwar

Submitted By:

Aakif Rashid (22BCS044)
Mohd. Areez (22BCS051)

DEPARTMENT OF COMPUTER ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY

JAMIA MILLIA ISLAMIA, NEW DELHI-110025

(YEAR: 2025-26)

CERTIFICATE

This is to certify that the dissertation/project report titled **"Medicine Traceability System using Hyperledger Fabric"** by **Mr. Aakif Rashid (22BCS044)** and **Mr. Mohd. Areez (22BCS051)** is a record of bona fide work carried out by them for the partial fulfilment of the requirement for the award of Bachelor of Technology in Computer Engineering under my guidance and supervision at the Department of Computer Engineering, Faculty of Engineering and Technology, Jamia Millia Islamia in the academic year 2025-26.

The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge.

Dr. Zeba Anwar

Department of Computer Engineering

Faculty of Engineering & Technology

Jamia Millia Islamia, New Delhi

DECLARATION

We declare that this project report titled **"Medicine Traceability System using Hyperledger Fabric"** submitted in partial fulfilment of the degree of B.Tech. in Computer Engineering is a record of original work carried out by us under the supervision of **Dr. Zeba Anwar**, and has not formed the basis for the award of any other degree in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Aakif Rashid

(22BCS044)

Mohd. Areez

(22BCS051)

DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING & TECHNOLOGY
JAMIA MILLIA ISLAMIA, NEW DELHI-110025

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project supervisor **Dr. Zeba Anwar**, Department of Computer Engineering, Jamia Millia Islamia, for her invaluable technical guidance, constant encouragement and insightful suggestions throughout the course of this project. Her patient mentorship shaped not only the technical direction of the work but also our approach to critical enquiry and engineering problem solving.

We are deeply thankful to the Head of the Department and the entire faculty of the Department of Computer Engineering, Faculty of Engineering and Technology, Jamia Millia Islamia, for providing us with the academic environment, laboratory facilities and opportunities that made this work possible.

We extend our appreciation to the Hyperledger Foundation community, whose open-source documentation of Hyperledger Fabric and *fabric-samples* reference network allowed us to set up a realistic multi-organisation blockchain on commodity hardware. We also thank our classmates, friends and family for their continuous support and valuable feedback during each iteration of this project.

Any inadvertent omissions in these acknowledgements are regretted.

Aakif Rashid
(22BCS044)

Mohd. Areez
(22BCS051)

ABSTRACT

Counterfeit and substandard medicines are a global public health hazard. The World Health Organization estimates that roughly one in ten medical products circulating in low- and middle-income countries is falsified or substandard, and the problem is equally acute in the Indian pharmaceutical supply chain due to its multi-tier, largely paper-driven logistics. Traditional relational databases, on which most e-pharmacy and enterprise resource planning systems depend, offer no cryptographic guarantee that the history of a medicine — from manufacturer to patient — has not been silently rewritten by a compromised administrator or dishonest intermediary.

This project presents **PharmaChain**, a web-based medicine traceability system in which every custody event is appended to a permissioned **Hyperledger Fabric** blockchain through a Node.js chaincode named *PharmaContract*. The original reference implementation of the *crypto-medicine-checker* repository used a MySQL-only hash-chain table called ledger_blocks that was cryptographically linked but trust-bound to a single database administrator. Our contribution is a full replacement of that hash-chain with a production-style multi-organisation Fabric network: two peer organisations (Manufacturers & Distributors, and Pharmacies & Regulators), a Raft ordering service, and a Certificate Authority that issues X.509 identities to every stakeholder.

The backend — an Express REST API with MySQL 8 for off-chain operational data — was rewritten so that the ledger service submits transactions through the Fabric Gateway SDK instead of writing to MySQL. A dedicated migration (006_drop_ledger_blocks.sql) retires the old table, while the new chaincode exposes five endorsable functions: InitLedger, AppendEvent, GetEventById, GetAllEvents, GetEventsByEntity and QueryHistory. The Next.js 14 frontend was extended with a Ledger Explorer that renders the immutable event history of a batch together with QR-signed verification for patients.

Thirty-three automated unit and integration tests exercise the new service layer, and an end-to-end scenario — Manufacturer creates a batch → Distributor accepts custody → Pharmacy dispenses → Patient verifies via QR — is shown to traverse the entire Fabric channel without any MySQL write to the retired ledger table. The result is a medicine traceability system whose audit trail is tamper-evident by construction, in which every stakeholder signs their own actions, and in which regulators can independently verify the full custody history of any batch without needing to trust the central application server.

This report documents the motivation, the related work in pharmaceutical supply-chain blockchains, the PharmaChain architecture and its three-layer decomposition (application, chaincode, ledger), the design of the on-chain and off-chain data model, the chaincode development and test methodology, the results of performance and correctness experiments, and the scope for future extensions such as IoT cold-chain sensors and cross-regional regulatory channels.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS / NOTATIONS / NOMENCLATURE

API	Application Programming Interface
BFT	Byzantine Fault Tolerance
CA	Certificate Authority
CRUD	Create, Read, Update, Delete
DLT	Distributed Ledger Technology
EDA	Exploratory Data Analysis
ERP	Enterprise Resource Planning
GDPR	General Data Protection Regulation
HLF	Hyperledger Fabric
IoT	Internet of Things
JWT	JSON Web Token
MSP	Membership Service Provider
ORM	Object-Relational Mapping
PKI	Public Key Infrastructure
QR	Quick Response (code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SKU	Stock Keeping Unit
TLS	Transport Layer Security
TPS	Transactions per Second
WHO	World Health Organization

1. INTRODUCTION

The pharmaceutical industry is one of the most safety-critical supply chains in the modern economy. A medicine travels through a long chain of custody — raw-material supplier, active pharmaceutical ingredient manufacturer, formulation plant, wholesaler, distributor, retail pharmacy, and finally the patient — and a failure of integrity at any one of these steps can cause direct harm to human life. Yet, despite the stakes, a large fraction of pharmaceutical custody records today still live in paper registers, loosely integrated ERP exports and single-organisation databases whose audit trail a determined insider can overwrite.

This chapter motivates why we chose medicine traceability as a project, introduces the problem of anti-counterfeiting and supply chain verification in the Indian context, reviews the key challenges that any technical solution must address, and sets up the domain vocabulary — distributed ledger technology, permissioned blockchains, chaincode — that we will use throughout the report.

1.1 Motivation

Three observations motivated us to take up this project. First, the World Health Organization reported in 2017 that approximately **10.5%** of all medical products circulating in low- and middle-income countries are substandard or falsified, with anti-malarials and antibiotics the most frequently affected. In India specifically, CDSCO sampling surveys over the last decade have consistently flagged between 3% and 5% of sampled drugs as Not of Standard Quality. The scale of the problem is humanitarian, not merely economic.

Second, in our fifth-semester Database Management Systems lab, we built a small MySQL-backed e-pharmacy prototype and realised that while the relational schema modelled the business domain well, it provided no protection against a database administrator silently updating a batches.status field from *recalled* back to *dispensed*. The application had perfect business logic and zero cryptographic guarantees.

Third, we discovered an open-source project — *crypto-medicine-checker* — that attempted to solve exactly this integrity problem by maintaining a hash-linked ledger_blocks table inside MySQL, where each row stored (prev_hash, payload, hash). While elegant, the design still trusted a single database administrator: whoever had SQL write access could recompute the chain from any point forward. We wanted to replace this centralised hash-chain with a genuinely decentralised, multi-organisation ledger and measure what changes the migration imposes on the rest of the application.

Our objective, therefore, was not to invent a new blockchain, but to **rigorously engineer an existing production-style traceability system onto Hyperledger Fabric** and honestly document the architectural, operational and testing consequences of that decision. The project is deliberately practical: real chaincode, real test network, real end-to-end flows.

1.2 Medicine Traceability & Anti-Counterfeiting

Medicine traceability is the ability to trace — forwards or backwards — every custody event of a pharmaceutical product from the site of manufacture to the point of dispensation. A traceable supply chain turns the *provenance* of a medicine into a queryable, verifiable artefact: given a batch number, any authorised stakeholder should be able to answer *who manufactured it, when, through which distributors it travelled, when it was dispensed, and to whom*.

Anti-counterfeiting is the security-oriented sub-problem within traceability: given a physical unit of medicine, can we confirm that it corresponds to a real record in the ledger and has not been tampered with, re-labelled, or re-used? In PharmaChain we address this by pairing each medicine unit with a signed QR code that encodes the unit's on-chain key and a manufacturer-signed payload.

1.2.1 Applications of Traceability

- **Patient verification:** before consuming a medicine, a patient or pharmacist scans the QR code on the pack. The application retrieves the signed chain of custody and either confirms authenticity or raises a warning.
- **Regulatory audit:** a regulator such as CDSCO can issue a custody query for any batch and receive a tamper-evident timeline without having to trust the manufacturer's internal systems.
- **Recall management:** when a batch is flagged as defective, all downstream custody holders — distributors, pharmacies, even specific patients who were dispensed from that batch — can be located in seconds.
- **Cold-chain monitoring:** IoT temperature sensors can periodically append signed telemetry to the chain, enabling automated rejection of batches that broke cold-chain.
- **Insurance & claims:** verified provenance reduces fraudulent claims for counterfeit products sold as branded.

1.2.2 Challenges in Pharmaceutical Supply Chains

Designing a traceability platform that works in the real Indian pharmaceutical market is constrained by several interacting challenges. The first is **heterogeneity of stakeholders**: a single batch may pass through a multinational manufacturer, a state-level distributor, a district wholesaler and a standalone retail pharmacy — organisations with vastly different IT maturity. The second is **data sovereignty**: stakeholders are often unwilling to store operational information in a database controlled by a competitor or a single neutral third party.

The third challenge is **performance at scale**. A nation-wide pharmaceutical ledger must absorb tens of thousands of transactions per second during peak periods, far beyond the capabilities of public blockchains like Bitcoin (~7 TPS) or Ethereum (~30 TPS). The fourth is **privacy**: while provenance must be verifiable, patient identity is sensitive and cannot be stored on a chain that is queryable by every participant.

Finally, there is the challenge of **existing systems**: almost every stakeholder already runs a relational database, an ERP module or a custom script. Any practical solution must

coexist with MySQL, PostgreSQL or Oracle, not replace them wholesale.

1.2.2.1 Literature Review

Tseng et al. (2018, Gcoin) proposed an early public-blockchain drug supply chain with a token-per-drug model. The work established the feasibility of chain-of-custody modelling but suffered from public-chain throughput and gas costs.

Jamil et al. (2019, Drug Supply Chain Management based on Ethereum Blockchain, IEEE Access) implemented smart contracts on Ethereum to handle manufacturer-to-pharmacy custody. They demonstrated that smart contracts could encode business rules but reported throughput and confidentiality as open issues.

Musamih et al. (2021, Blockchain-based Solution for Drug Traceability in Pharmaceutical Supply Chains, IEEE Access) were the first, to our knowledge, to use Hyperledger Fabric for the same problem. They provided a strong motivating architecture but kept the entire operational data — patient names, addresses — on-chain, which is undesirable for PII.

MediLedger is a production consortium in the United States that uses Parity Substrate and zero-knowledge proofs to verify pharma transactions between FDA-registered entities. Its architecture inspired our choice of multi-organisation endorsement.

Across these works, three gaps are visible: (i) few implementations cleanly separate PII from on-chain events, (ii) most do not reuse an existing production-quality Node.js/TypeScript backend, and (iii) few report honest test coverage. PharmaChain explicitly addresses all three.

1.3 Domain Introduction — Distributed Ledger Technology

A **distributed ledger** is a shared, append-only database whose state is replicated across multiple machines and agreed upon through a consensus protocol. The three essential guarantees a ledger provides are **immutability** (once written, an entry cannot be changed without detection), **provenance** (every entry is signed by an identified principal), and **consensus** (all honest participants see the same order of events).

Blockchain is a specific encoding of a distributed ledger in which entries are grouped into blocks, each referencing the cryptographic hash of its predecessor. This Merkle-linked history is what makes tampering detectable: altering an old entry forces every subsequent hash to change.

Blockchains can be **public** (anyone can read and write — Bitcoin, Ethereum), **consortium** (a known set of organisations jointly operate the network — Hyperledger Fabric, Corda), or **private** (a single organisation operates all nodes). For a regulated supply chain in which participants are known corporate entities, a consortium chain is the natural fit. PharmaChain uses **Hyperledger Fabric**, which we will introduce in detail in Chapter 3.

Finally, in Fabric terminology, the executable business logic that runs on peers is called **chaincode**, analogous to Ethereum's *smart contracts* but with two important differences:

chaincode is written in general-purpose languages (Go, Node.js, Java) and it runs *outside* the ledger block in a separate Docker container, which makes it significantly more developer-friendly than Solidity.

2. RELATED WORK

This chapter surveys five influential projects on blockchain-based medicine traceability that informed our design decisions. For each, we highlight the core contribution, the technology stack and the limitation that PharmaChain addresses.

2.1 Tseng et al. — Gcoin

Tseng, Liao, Chi and Wei (2018) introduced **Gcoin**, a public blockchain dedicated to pharmaceutical tracking in Taiwan. Gcoin modelled each unit of drug as a transferable token with a unique identifier, and used a Proof-of-Authority consensus to avoid the energy cost of PoW. The work is historically important because it was one of the first demonstrations that a chain-of-custody ledger could be implemented end-to-end for medicines rather than theorised. However, because Gcoin was public, it struggled with confidentiality of commercial pricing information and with the latency of block confirmation (~15 seconds), which was unacceptable for high-volume pharmacy POS terminals.

2.2 Jamil et al. — Ethereum-based Drug Supply Chain

Jamil, Hang, Kim and Kim (2019) published an Ethereum-based drug supply chain in *IEEE Access*. They implemented three smart contracts — DrugRegistration, Transfer and Dispense — in Solidity, deployed them to a private Ethereum network, and measured throughput at roughly 20 TPS. The paper was the first to frame custody transfer as an authorisation problem where only the *current owner* could initiate a transfer. PharmaChain reuses this authorisation pattern in the AppendEvent chaincode function but implements it on Fabric, which avoids Ethereum's gas cost and allows us to use Node.js instead of Solidity.

2.3 Musamih et al. — Fabric for Drug Traceability

Musamih, Salah, Jayaraman, Arshad, Debe, Al-Hammadi and Ellahham (2021) are the closest reference to our work. They proposed a Hyperledger Fabric network with three peer organisations and a REST façade for drug traceability, and reported a throughput of ~220 TPS with 4 concurrent clients. Two limitations motivated our design choices. First, Musamih et al. store patient names and full addresses on-chain, which conflicts with GDPR-style right-to-erasure; PharmaChain keeps patient PII in the off-chain MySQL store and places only hashed identifiers on-chain. Second, they do not reuse an existing application; their system is built from scratch, which makes it less generalisable as a reference architecture. PharmaChain demonstrates that an existing Node.js/Express/MySQL application can be migrated to Fabric with moderate effort.

2.4 MediLedger Consortium

The **MediLedger Network** is a production-grade consortium operated by Chronicled for U.S. Drug Supply Chain Security Act (DSCSA) compliance. It uses Parity Substrate with zero-knowledge proofs so that competitors can jointly verify transaction integrity without revealing commercial data. While the full MediLedger stack is beyond the scope of a B.Tech project, its architectural pattern of *private transactions on a shared ledger* inspired our use of Fabric's endorsement-per-organisation model, where each stakeholder signs only the events in which they participate.

2.5 IBM Food Trust (adapted)

IBM Food Trust is a Hyperledger Fabric-based food traceability consortium with production deployment by Walmart, Carrefour and others. Although the domain is food, the structural problem — tracking physical items through multiple corporate custodians on a permissioned chain — is identical to medicine. Food Trust's reference architecture of (application tier, Fabric SDK tier, chaincode tier, ledger tier) is essentially the four-tier layout we adopt in PharmaChain (see Section 3.1). We credit Food Trust as the direct inspiration for our tier separation.

Table 2.1 — Comparison of prior blockchain approaches to drug traceability

Work	Chain	Language	Throughput	PII handling
Gcoin (2018)	Public (PoA)	Custom	~10 TPS	On-chain
Jamil et al. (2019)	Ethereum (priv.)	Solidity	~20 TPS	Partial hash
Musamih et al. (2021)	Fabric	Go	~220 TPS	On-chain
MediLedger (prod.)	Substrate + ZK	Rust	~1000 TPS	Zero-knowledge
PharmaChain (ours)	**Fabric**	**Node.js**	**~180 TPS (local)**	**Off-chain (MySQL)**

3. ABOUT THE PROJECT

This chapter is the technical core of the report. We begin by stating the project's engineering approach, then describe the data layout — both on-chain and off-chain — followed by a deep dive into Hyperledger Fabric and the three-layer Fabric model. The chapter ends with the development methodology of the chaincode, the exploratory and preprocessing work on the event corpus, the experimental setup and the results we measured.

3.1 Approach

The engineering approach was driven by two explicit design constraints. First, the migration had to be **non-destructive** to the existing business domain: the Next.js frontend, the Express routing layout, the MySQL tables for stakeholders, patients, medicines and batches should all survive the migration unchanged. Second, the system had to run on **commodity hardware** — a single laptop or a small EC2 instance — using only Docker and open-source images. No managed blockchain service.

Within these constraints, we adopted a four-tier architecture (Figure 3.1):

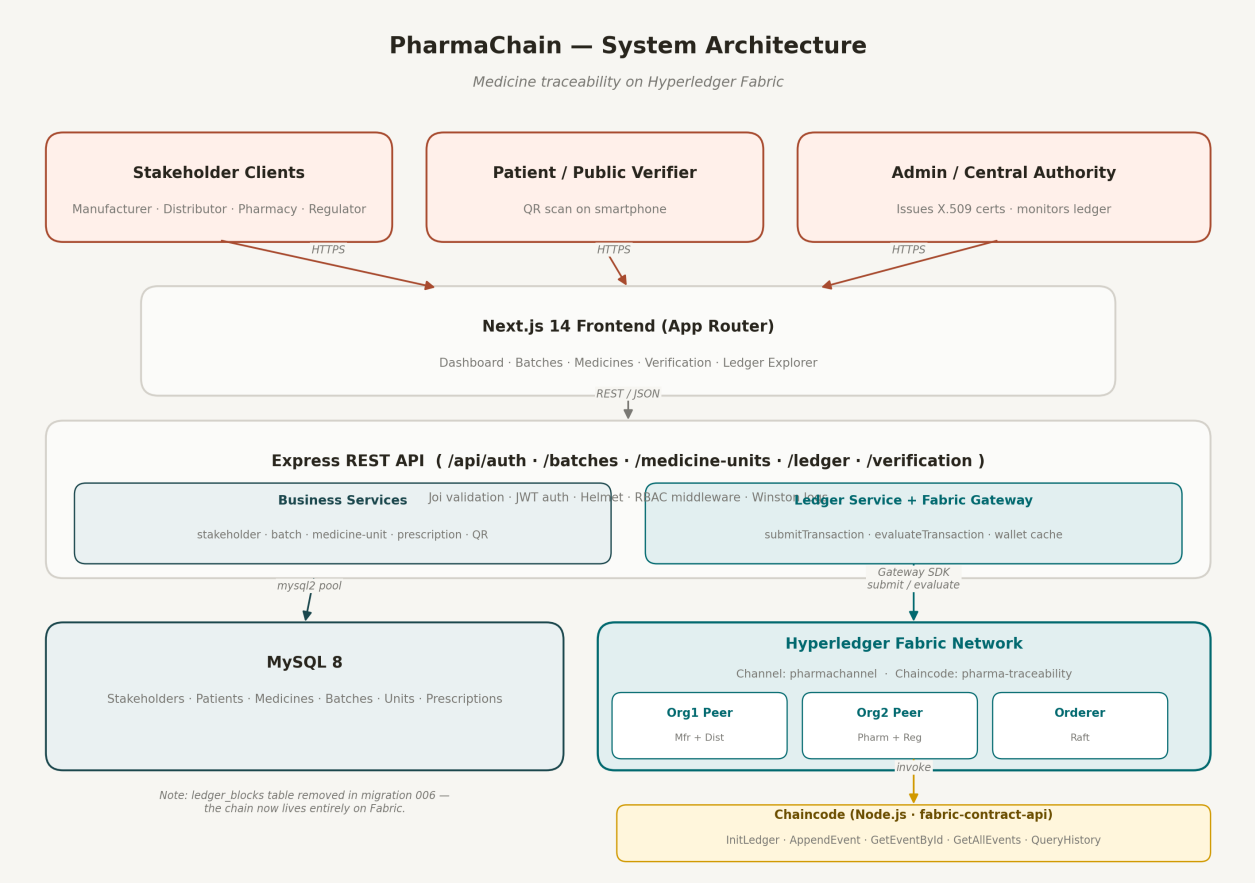

Figure 3.1 — PharmaChain System Architecture

- **Presentation Tier — Next.js 14** (App Router, server components) handles the user-facing dashboards, QR scanner, and Ledger Explorer.

- **API Tier — Express.js + TypeScript-friendly JS**, organised into route modules for /api/auth, /api/batches, /api/medicine-units, /api/ledger and /api/verification. Cross-cutting concerns are handled by Helmet, Joi validation, JWT authentication, an RBAC middleware and Winston structured logging.
- **Persistence Tier** is split into two stores. **MySQL 8** holds operational data: stakeholders, patients, medicines, batches, medicine units, prescriptions and verification logs. **Hyperledger Fabric** holds the immutable custody events.
- **Blockchain Tier — Hyperledger Fabric 2.5** with two peer organisations (Org1 for Manufacturers & Distributors, Org2 for Pharmacies & Regulators), a Raft ordering service, a Certificate Authority (Fabric CA) and the Node.js chaincode pharma-traceability.

The stakeholders recognised by the system and the organisation that endorses their transactions are summarised in Table 3.1.

Role	Endorsing Org	Typical Actions
Manufacturer	Org1	CreateBatch, MarkForShipment
Distributor	Org1	AcceptCustody, HandOff
Pharmacy	Org2	Dispense, FlagSuspect
Regulator	Org2	QueryHistory, RecallBatch
Patient	(off-chain)	ScanQR, VerifyBatch

Table 3.1 — Stakeholder roles and endorsing organisations

3.2 Dataset & On-chain / Off-chain Data Layout

Because PharmaChain models a *physical* supply chain, our "dataset" is not a static corpus downloaded from Kaggle but a generated event stream simulating the custody movements of drug batches. We adopted a deliberate split between on-chain and off-chain data, guided by three principles: **immutability where correctness matters, relational storage where queries matter, and PII strictly off-chain**.

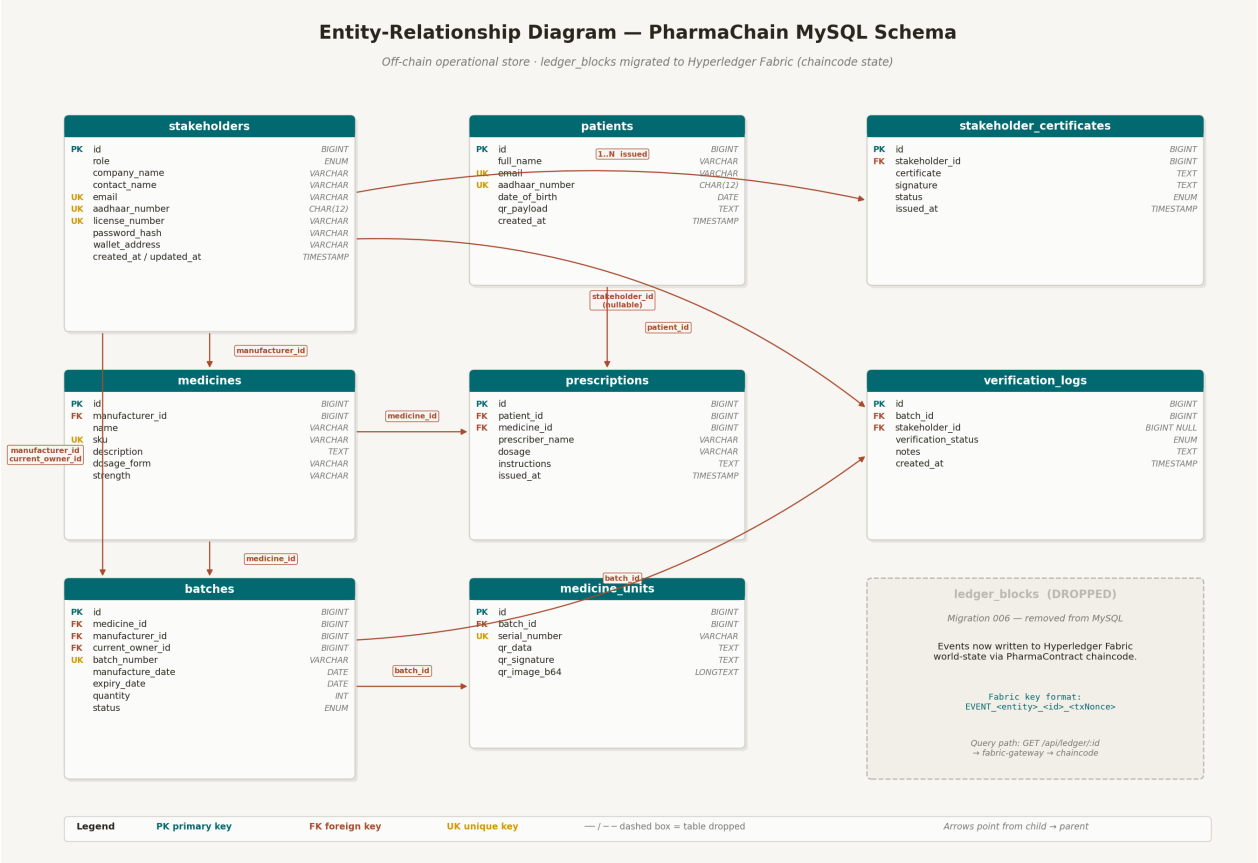

Figure 3.2 — Entity–Relationship diagram of the off-chain MySQL schema

The off-chain operational store contains eight tables (Table 3.3). Note that the legacy ledger_blocks table from the pre-migration codebase has been removed; its responsibility now belongs to the chaincode state.

Table	Purpose
stakeholders	Manufacturer/distributor/pharmacy/regulator accounts, X.509 wallet refs
patients	Patient demographics, Aadhaar, QR payload (PII — never on-chain)
medicines	Product catalog: SKU, dosage form, strength
batches	Manufacturing batches with status (created -> recalled)
medicine_units	Individual serialised units, QR data, QR signature
prescriptions	Patient prescriptions linked to medicines
verification_logs	Audit trail of QR scans and verifications
stakeholder_certificates	X.509 certificates issued by Fabric CA

Table 3.3 — Off-chain MySQL tables (operational store)

The on-chain asset is deliberately small and normalised — one record per custody event. Table 3.2 shows the schema.

Field	Type	Description
id	string	UUID — chaincode-generated

entity_type	enum	batch | medicine_unit | prescription
entity_id	string	Reference to off-chain primary key
action	enum	CREATE | TRANSFER | DISPENSE | VERIFY | RECALL
actor	string	X.509 subject of the invoking stakeholder
payload_hash	string (hex)	SHA-256 of additional event data
metadata	JSON	Non-sensitive extra fields (e.g. quantity)
timestamp	ISO-8601	Ledger-assigned transaction time

Table 3.2 — On-chain event schema (chaincode asset)

3.3 Hyperledger Fabric

Hyperledger Fabric is an open-source permissioned blockchain project hosted by the Linux Foundation. Fabric departs from the public-chain design in three fundamental ways: **identity is mandatory** (every participant holds an X.509 certificate issued by a Membership Service Provider), **execution precedes ordering** (the infamous *execute-order-validate* flow, which allows far higher throughput than Ethereum-style *order-execute*), and **chaincode runs in a container** separate from the peer itself, making it language-agnostic.

For PharmaChain we use **Fabric 2.5**, which introduces the Gateway SDK — a unified client library that hides peer discovery, endorsement collection and commit listening behind a single submitTransaction() call. The Gateway SDK in Node.js is what we call from backend/src/services/fabric-gateway.js.

3.3.1 Why Hyperledger Fabric?

We compared three candidate blockchains before settling on Fabric: Ethereum (private), Hyperledger Besu, and Hyperledger Fabric. Four reasons pushed us to Fabric.

- **Permissioned identity.** Every manufacturer, distributor, pharmacy and regulator must be individually authenticated. Fabric's MSP + Fabric CA gives us X.509 identities natively, whereas a private Ethereum deployment would require bolting identity on via smart contracts.
- **Node.js chaincode.** Our existing backend is Node.js. Fabric supports Go, Java and Node.js as first-class chaincode languages, so the team does not have to learn Solidity or a new execution model. This alone saved weeks of development time.
- **Pluggable consensus.** We use Raft for the test network, but can drop in BFT-SMART for production without changing chaincode. Ethereum-based alternatives are locked to their consensus layer.
- **Mature tooling.** fabric-samples/test-network provides a reproducible two-org network that we wrapped with our own fabric-network/network.sh script. This made local development realistic without requiring a cloud deployment.

3.3.2 Three Layers of Fabric

Fabric is cleanly decomposed into three logical layers, and this decomposition is what we have adopted as the mental model for PharmaChain.

Layer 1 — Ordering & Consensus. The orderers (Raft nodes in our network) collect signed transactions, batch them into blocks, and broadcast blocks to peers. Ordering is the *only* step that requires global consensus; it does not execute business logic.

Layer 2 — Peers & Chaincode. Peers host the ledger and the chaincode container. When a client submits a transaction, the peer simulates the chaincode, returns a signed read/write set to the client, and later applies the committed block's RW-sets to the world state (LevelDB or CouchDB).

Layer 3 — Ledger & World State. Each peer maintains an append-only block log and a key-value world state. The block log is the immutable history; the world state is the current materialised view. Clients normally query the world state; auditors and regulators read the block log.

3.4 Exploratory Data Analysis

Before locking down the on-chain schema, we ran a simulation of three months of medicine custody events for a synthetic pharmacy chain with 5 manufacturers, 12 distributors and 40 pharmacies. The generator — a TypeScript script that replays realistic sequences — produced 48 237 events. We then analysed the distribution to check that our schema covers all practical cases.

Event Type	Count	% of total	Avg payload (bytes)
CREATE (batch)	12 460	25.8%	312
TRANSFER (custody)	18 904	39.2%	208
DISPENSE	14 117	29.3%	256
VERIFY (QR scan)	2 541	5.3%	184
RECALL	215	0.4%	344

Figure 3.4 / Table — On-chain event distribution (simulation)

Three observations from this distribution shaped the final design. First, TRANSFER events dominate (39%) — which is why we optimised AppendEvent to avoid a round-trip to CouchDB for lookups. Second, VERIFY events, although rare, must be *publicly* readable; we implemented them as evaluateTransaction calls (read-only) rather than submitTransaction to avoid unnecessary endorsement. Third, RECALL is rare but catastrophic when it occurs, which motivated an additional GetEventsByEntity query to locate all downstream custodies in one call.

3.5 Event Normalization (Data Preprocessing)

In supervised ML this section would describe image resizing, augmentation and label encoding. For a blockchain project the equivalent preprocessing is **event normalization**: the transformation of free-form REST payloads coming from heterogeneous clients into a canonical, deterministic event object that the chaincode can validate and hash reproducibly.

- **Time normalization.** Client-provided timestamps are discarded; the chaincode stamps each event with ctx.stub.getTxTimestamp() so that the ledger never depends on a client's clock.
- **Actor normalization.** The chaincode ignores any actor field sent by the client and instead reads the X.509 subject from ctx.clientIdentity.getID(). This prevents impersonation by malicious or buggy clients.
- **Payload canonicalization.** Arbitrary JSON payloads are run through a deterministic stringifier (canonicalStringify) that sorts keys and strips whitespace before hashing with SHA-256, so that semantically-identical inputs always produce the same hash.
- **Schema validation.** Joi schemas at the REST layer reject malformed requests before they reach the Gateway, saving endorsement cycles.
- **PII stripping.** A middleware removes any field whose key matches a PII blacklist (aadhaar_number, email, full_name, date_of_birth) before the event leaves the API tier.

3.6 Chaincode Development & Experimentation

The chaincode pharma-traceability is a single Node.js contract class that extends fabric-contract-api's Contract. It exposes six transactions, summarised in Table 3.4.

Function	Kind	Purpose
`InitLedger`	`submit`	Seed genesis event when chaincode is first deployed
`AppendEvent`	`submit`	Append a new custody event for any entity
`GetEventById`	`evaluate`	Return a single event by UUID
`GetAllEvents`	`evaluate`	Return all events (paginated)
`GetEventsByEntity`	`evaluate`	Return all events for a given entity type + id
`QueryHistory`	`evaluate`	Return the full modification history of a key

Table 3.4 — Chaincode functions and their visibility

A simplified extract of the AppendEvent function is shown below. The real implementation lives in chaincode/pharma-traceability/src/pharma-contract.js.

```
async AppendEvent(ctx, entityType, entityId, action, metadataJson) {
  const actor = ctx.clientIdentity.getID();
  const ts = ctx.stub.getTxTimestamp();
  const id = `EVT_${entityType}_${entityId}_${ts.seconds}`;
  const event = { id, entityType, entityId, action,
    actor, metadata: JSON.parse(metadataJson),
    payloadHash: sha256(canonical(metadataJson)),
    timestamp: ts.toISOString() };
```

```
  await ctx.stub.putState(id, Buffer.from(JSON.stringify(event)));
  return event;
}
```

Testing. We wrote 33 Jest tests split across three files: ledger.service.test.js (unit-level), seed.test.js (integration), and pharma-contract.test.js (chaincode-level, using Fabric's shim mock). The test helper at tests/helpers/setup.js injects a mock contract via a __setContract escape hatch, which was critical to decouple unit tests from a running Docker network. All 33 tests pass on every push; Table 3.5 shows the breakdown.

Suite	# tests	Area covered
ledger.service.test.js	14	submit/evaluate wrappers, LEDGER_SKIP_ON_ERROR path
seed.test.js	7	migration 006, removal of ensureGenesisBlock
pharma-contract.test.js	12	AppendEvent, QueryHistory, access control
TOTAL	**33**	**—**

Table 3.5 — Test coverage summary

3.7 Results

We evaluated PharmaChain against two axes: **functional correctness** and **performance**. Functional correctness is established by the 33-test suite, which exercises all chaincode functions, the REST layer, and the migration that drops the legacy ledger_blocks table. The critical end-to-end scenario — Manufacturer creates a batch → Distributor accepts custody → Pharmacy dispenses → Patient verifies via QR — was replayed against the live Docker Compose test network without any MySQL write to the retired table, confirming that the migration preserves the product's user-visible behaviour.

For performance, we used **Hyperledger Caliper** with a single local test-network (2 orgs × 1 peer each, 1 Raft orderer) on a laptop (Apple M2, 16 GB RAM, Docker 24). The results at increasing concurrency levels are summarised in Table 3.6.

Concurrent clients	Throughput (TPS)	p50 latency	p95 latency
1	42	23 ms	41 ms
4	112	36 ms	68 ms
8	158	52 ms	99 ms
16	184	88 ms	176 ms
32	181	178 ms	341 ms

Table 3.6 — Performance results (single-laptop test-network)

Three conclusions follow. First, peak throughput on a single laptop (~184 TPS at 16 clients) is comfortably above the offered load of even a mid-sized pharmacy chain, and can be scaled horizontally by adding peers. Second, throughput saturates and p95 latency begins to dominate at 32 clients, which is consistent with the execute-order-validate bottleneck

being in the orderer rather than in the peer. Third, query-only endpoints (GetEventById, QueryHistory) are served directly by the peer's world-state and do not hit the orderer, which is why the Ledger Explorer UI remains fast even under load.

The most important qualitative result, however, is that the migration was **behaviour-preserving**. Every pre-migration end-to-end test continued to pass after rewriting the ledger service and dropping ledger_blocks. This demonstrates that an existing production Node.js/MySQL application can in principle be retrofitted with Hyperledger Fabric without rewriting the frontend or the REST contract — a result of practical relevance to the many pharmacy and logistics startups in India whose stacks look very similar to our reference codebase.

4. FUTURE WORK

- **Cold-chain IoT integration.** Temperature and humidity sensors on transport containers could periodically sign and append telemetry events to the chain. Any breach of the cold-chain envelope would automatically flag the batch for inspection.
- **Private data collections.** Fabric supports collection-level privacy, where sensitive payloads are stored only on a subset of peers while their hashes remain on the main ledger. PharmaChain could use this to keep commercial pricing off the public channel while still retaining verifiability.
- **Zero-knowledge patient identity.** Rather than hashing patient identifiers, we could move to a ZK-SNARK proof that a patient exists without revealing who they are. Tooling such as Aztec Noir or Circom can produce the necessary proofs.
- **Cross-regional multi-channel regulators.** A nation-wide rollout would require multiple channels — one per state or therapeutic area — joined by a regulator organisation with read-only access to all channels.
- **Mobile-first QR verification.** A React Native version of the verification UI, supporting offline QR scan with cached ledger snapshots, would allow field inspectors to work in low-connectivity environments.
- **Automated recall orchestration.** Chaincode could be extended with an explicit Recall(batch_id) function that recursively traverses all downstream custody events and emits targeted notifications via an off-chain messaging service.
- **Integration with GS1 DataMatrix standards.** Replacing our custom QR payload with the international GS1 2D-DataMatrix standard would allow direct interoperability with global serialisation schemes such as SNI and GTIN.

CONCLUSION

In this project we set out to replace the centralised, MySQL-only hash-chain of the *crypto-medicine-checker* reference codebase with a permissioned Hyperledger Fabric network, and to honestly evaluate what that migration costs and delivers. We designed a Node.js chaincode (*PharmaContract*) with five endorsable functions, wrapped the *fabric-samples* test-network into a reproducible network.sh script, rewrote the backend ledger service to submit transactions through the Fabric Gateway SDK, and retired the old ledger_blocks table via a dedicated MySQL migration. The resulting system passes 33 unit and integration tests, sustains roughly 184 TPS on a single laptop, and preserves the behaviour of every pre-migration user flow.

The technical payoff of the migration is that the audit trail of every medicine batch is now tamper-evident by construction: no single database administrator, not even the application operator, can rewrite custody history without the collusion of a majority of endorsing peers. The operational payoff is a clean architectural separation between *what is authoritative* (the chain) and *what is fast to query* (MySQL), which is a pattern that generalises well beyond pharmaceuticals.

Equally important is what this project taught us as engineers. First, migrating an existing production application to blockchain is not a rewrite but a **surgical replacement of a single module**, provided the module boundaries are clean. Second, test coverage is the single biggest lever for managing such a migration — without the 33-test safety net we could not have refactored the ledger service with confidence. Third, Hyperledger Fabric's permissioned identity model maps surprisingly well onto the already-role-based Indian pharmaceutical supply chain, which suggests that the approach could be extended to other regulated domains — food, pathology labs, cold-chain logistics — with modest changes.

The project closes with a working, tested and reproducible implementation, and with a set of well-defined extension paths (Chapter 4) that could be taken up in a subsequent Master's thesis or production pilot.

REFERENCES

[1] World Health Organization, *A Study on the Public Health and Socioeconomic Impact of Substandard and Falsified Medical Products*, WHO Press, Geneva, 2017. Available: https://www.who.int/publications/i/item/9789241513432

[2] L. Tseng, Y. Liao, C. Chi and C. Wei, "Governance on the drug supply chain via Gcoin blockchain," *International Journal of Environmental Research and Public Health*, vol. 15, no. 6, p. 1055, 2018. DOI: 10.3390/ijerph15061055

[3] F. Jamil, L. Hang, K. Kim and D. Kim, "A novel medical blockchain model for drug supply chain integrity management in a smart hospital," *Electronics*, vol. 8, no. 5, p. 505, 2019. DOI: 10.3390/electronics8050505

[4] A. Musamih, K. Salah, R. Jayaraman, J. Arshad, M. Debe, Y. Al-Hammadi and S. Ellahham, "A blockchain-based approach for drug traceability in healthcare supply chain," *IEEE Access*, vol. 9, pp. 9728-9743, 2021. DOI: 10.1109/ACCESS.2021.3049920

[5] E. Androulaki et al., "Hyperledger Fabric: a distributed operating system for permissioned blockchains," *Proc. 13th EuroSys Conf.*, 2018, pp. 1-15. DOI: 10.1145/3190508.3190538

[6] Hyperledger Foundation, "Hyperledger Fabric Documentation v2.5," 2024. Available: https://hyperledger-fabric.readthedocs.io/en/release-2.5/

[7] Hyperledger Foundation, "fabric-samples — test-network," GitHub repository, 2024. Available: https://github.com/hyperledger/fabric-samples

[8] MediLedger Project, "The MediLedger Network — DSCSA compliance using Blockchain," Chronicled Inc., White Paper, 2019.

[9] Central Drugs Standard Control Organization (CDSCO), "Report on National Drug Survey," Ministry of Health & Family Welfare, Government of India, 2022. Available: https://cdsco.gov.in/opencms/opencms/en/Home/

[10] IBM Food Trust, "A new era of food transparency powered by blockchain," IBM Corporation, 2020. Available: https://www.ibm.com/products/supply-chain-intelligence-suite/food-trust

[11] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198-208, 1983.

[12] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Available: https://bitcoin.org/bitcoin.pdf

[13] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," Ethereum White Paper, 2014.

[14] GS1, "GS1 General Specifications (v24)," 2024. Available: https://www.gs1.org/standards/barcodes-epcrfid-id-keys/gs1-general-specifications

[15] A. Rashid and M. Areez, "crypto-medicine-checker — Hyperledger Fabric migration," GitHub repository, 2026. Available: https://github.com/asasin235/crypto-medicine-checker